IMY 220 Project 2026

Photo sharing website

The project must be completed by Thursday **29 October**.

We recommend you constantly refer to **this spec as well as your individual deliverable specs** to make sure your project meets all of the requirements by the end.

Make sure you also **continuously test** your project as you add new functionality to make sure everything keeps working as expected.

Contents

Objective

Create a photo sharing website that provides an interface for sharing and saving photos. You must use the technologies and techniques as specified in the project spec section to write your own code.

You must use the technologies and techniques specified in the project spec section to write your own code.

Your website must have a unique visual design that fits with the theme of the website (i.e. a photo sharing website). You are not allowed to simply use another website's design or create a "knockoff" of another website when designing the visual theme of your site. Note that you also don't have to use the terminology used in this document to describe functionality, such as "post", "friend", "album", etc. You may name these however you want based on the theme of your website.

The purpose of the project is for you to demonstrate the skills that you have learned in IMY 220 - **not anything else**. Therefore, you will **NOT** get marks for proving your proficiency with any other skills OR for using code that is not your own (e.g., from other libraries or templates).

A Note on AI Use

AI-generated content is **NOT** allowed as a substitute for your own code. **Do not use AI to generate code for your project**. The scope of the project is too large to generate code that is reliable / functional. You will also find it difficult to understand how your code works, which will be detrimental as you **will be required to explain** parts of your project during your deliverable demo sessions. If you are unable to explain how your code works you will get zero for that section.

Submission Instructions

- ***Late projects will not be accepted.***
- ***Email project submissions / Google Drive links will not be accepted.***
- You are required to create a **GitHub repository** to store all of your project files and have some form of source control. The repository must be set to **public** so your code can be marked. The link for this repository should be submitted on **ClickUP** along with the relevant Docker files for each of the deliverables. You will still be required to submit all of your project files for the final deliverable as well as the GitHub link.
- Include a **readme.txt** that includes all commands that are required to build and run your project, as well as a basic explanation of your directory structure.
- You have to upload your files to **GitHub** as well as **ClickUP**. The markers should be able to download from either GitHub or ClickUP and it should work the same.
- ***IMPORTANT****: Please do not move your files around to different directories (in order to fix the directory structure) after you have finished writing your code. This can result in your code breaking.*

ClickUP

The submission link on ClickUP will close at **exactly 23:59 on Thursday 29 October**.

- **Do not wait** until just before 23:59 to upload because your submission might take longer than usual to upload.
- Instructions:
    - Place: **all your project files**, along with your Docker files and your exported MongoDB database in a folder named with your Position_Surname. Your Position can be found on ClickUP.
    - Compress the **FOLDER** using a zip archive format.
    - Ensure that any **node_modules** folders are **NOT** included in your ZIP folder.
    - Upload the compressed folder (**Position_Surname.zip**) to ClickUP to the relevant project link.

IMPORTANT: Do not expect the lecturer to immediately respond to communications around the submission time. It is your responsibility to make sure you submit all the relevant files well before the submission time.

Project Demo

You will have to demo your project during the demo sessions to get marks for it.

- You will have to demo using either the latest version of either Google Chrome or Mozilla Firefox.
- When you demo your project, you will be required to show how you have implemented all the instructions by demonstrating the functionality **on the website itself** (not just in the database).
    - You will receive marks for achieving working functionality according to the instructions. For example, if you are required to implement functionality that modifies and displays database data on the website itself, you will receive no marks for simply modifying the database.
- Demo sessions will be made available on ClickUP closer to the deadline.
- Demo sessions will take place on **Monday 2 November** and **Tuesday 3 November**.
- Project demos will take place in the **Immersive Technology Lab on the 4th floor of the IT building (IT 4-62)**.
- ***You will receive no marks if you do not demo your project.***

*You will **lose marks** in the following conditions*

- If your system does not have all the required functionality.
- If you use any other technology or technique that is not part of the course.
- If you use code that is not your own, i.e., if you use code generators or templates, you will not get marks for it.

*You will get **no marks** in the following conditions*

- If you submit work that is not your own.
- If you do not upload a Dockerfile / Docker Compose file.
- If the Dockerfile / Docker Compose file does not run the application.
- If you do not upload to ClickUP.

Main pages

All pages must clearly display the name of the website creatively and have a unique visual design.

Splash page

- When an unregistered user, or a user who is not logged in, visits the website, they must only be able to access the "splash page".
    - *A splash page is the first webpage a user sees before being given access to the main content of the website.*
- The splash page should inform new users about the purpose of the website and the services it provides. Include a tagline or other introductory content that explains the website's purpose.
- The splash page is the only page that does not have the same consistent layout as the rest of the webpages.
- The splash page must contain at least the name of the website as well as a log in and registration form.
    - Users must be able to register using their email address.
- ***BONUS****: catch new users' attention and show off your design skills by creating a well-designed landing page that explains the goal of your website. A landing page is generally more detailed compared to a simple splash page and is meant to sell the service you are offering.*

  Divide this page into well-designed sections and use the page scroll to trigger visual effects when the user scrolls down. How you make use of scrolling effects is entirely up to you.

  (You can find examples of this here: https://www.awwwards.com/inspiration_search/?text=Scrolling)

Home page

- When a user is **logged in**, the home page must display that user's local activity feed in reverse chronological order.
- Users must be able to switch between their local activity feed (displaying activity from their friends) and a global activity feed (displaying activity from all users).
- By default, all activity must be sorted by creation date in reverse chronological order. Users should also be able to sort the activity feed in another meaningful way, for example, by popularity based on the number of comments.
    - ***BONUS:*** *Allow users to combine sorting and filtering, e.g., "Most commented this week", "Newest posts containing a selected hashtag", or "Only posts from friends sorted by popularity."*

Profile page

- A user should be able to go to their profile page when they are logged in.
- The profile must have a profile image.
    - ***BONUS**: allow users to add an image using drag-and-drop.*
    - ***BONUS**: come up with a way to randomise the placeholder image so that each one is different in some way. This should go beyond simply having a few images to select from.*
- A profile must display appropriate personal information about a user, e.g., name, username (if appropriate), pronouns, short bio, links to other social media, etc. Look at other platforms/websites for ideas/guidance on what to include.
  Note: Since all this information will be publicly visible, no private information must be shown.
- Each user's list of friends should appear on their profile but should only be visible if the logged-in user is friends with the user whose profile they are viewing. (Users should also be able to view their own list of friends on their profile.)
- Users must be able to edit the appropriate information on their profile in a logical and intuitive manner.
- When you visit another user's profile, it must clearly indicate your current friendship status (e.g., friends, friend request pending, or not friends).
    - Any user can send another user a friend request, which must be accepted before they become friends. Once two users are friends, each user's posts will appear in the other's local activity feed and they can view each other's list of friends on their profiles.
    - ***BONUS**: Allow users to mark other users as favourites so that their posts appear first in the activity feed.*
- There must be a logical way to manage friendships between users.
- All the user's posts and albums must appear on their profile page. You must come up with a sensible and visually appealing way to display this. Keep in mind that there might be many posts and many albums with many images inside each, which means you must come up with an intuitive and visually appealing way to provide access to each.

Activity and projects

The website must provide two activity feeds: a local activity feed and a global activity feed. Activity in this case refers to creating new posts or new albums and to adding new items to an album.

The local activity feed consists of all the activity for the current user as well as for all their friends, while the global activity feed consists of all the activity for all users on the website. Each activity item must display the relevant image(s) together with appropriate information describing the activity that took place. For posts, this should include the accompanying text and hashtags where applicable. Activity should be displayed in an **informative and intuitive way**, e.g., "[user] added 2 photos to the album [album name]" with the relevant information and album's associated image.

Activities must be displayed in an **aesthetically appealing way**. Simply displaying activity in Bootstrap cards stacked underneath one another is **not** sufficient; find inspiration from other websites or applications that make use of news feeds or activity feeds.

Posts

A post in this context refers to an image with a text description, and one or more hashtags. The following applies with regards to creating and editing posts:

- Any user can create a post, making them the owner of that post. When creating a post, the user must be able to:
    - Add an image, text description, and hashtags. You can decide whether users should be able to add them to the text description or in a separate input.
      Note: A hashtag is an interactive piece of text that is created by prefixing a predefined symbol to it, e.g., #lunchtime. When a user clicks on a hashtag, it should behave as though they performed a search for that hashtag, displaying all posts containing that hashtag.
    - ***BONUS**: allow users to add images using drag-and-drop.*
- The post's information must be displayed both in the activity feed and on a dedicated post page. The post page is functionally similar to a profile page, but displays the post, its description, hashtags, comments, and the album(s) to which it belongs, if any (the album information is not displayed in the activity feed).
- The creator of a post must be able to edit the post's description and hashtags, but not its image.
- Other users must be able to comment on a post. At least some of the comments should be displayed with the post in the activity feed.
    - These should be displayed in a logical way, for example, if there are 50 comments on a post, only a few of them should be displayed on the newsfeed and the rest should be accessible by going to the post's page.
- Other users should be able to report a post (users cannot report their own posts). A user who reports a post should be able to select a reason for doing so from a list of predefined reasons (only admins are able to create new reasons for reporting posts). You must come up with a sensible interface for implementing this.
- The creator of a post must also be able to delete their posts. This should automatically delete all comments that were posted for the post being deleted.

    - ***BONUS**: when the deleted post is the only post in an album, the album should automatically be deleted as well.*

Albums

The following requirements apply to creating and managing albums:

- Any user can create albums, which are collections of posts. There must be a sensible and intuitive way for users to create albums and add posts to them.
    - An album can have a name and description, both of which can be changed by the user who created the album.
    - An album can also have hashtags, which should function similarly to those of posts.
    - Users should be able to change the album's name, description, and hashtags.
    - Users should be able to add or remove posts from their own album.
    - ***BONUS**: When adding hashtags to an album, use the list of hashtags contained by the posts inside an album and provide them as recommended hashtags to add. Hashtags that appear*

more frequently should appear first in the recommendation list and the list of recommended hashtags should not contain duplicates of hashtags. You must come up with an aesthetically pleasing and intuitive way to provide this functionality.

- Users can delete their own albums. This does **not** delete the posts inside the album.

Reported posts

- If a post has been reported more than twice, it must be hidden from the activity feed and replaced with an appropriate message informing users that the post has been reported. Users must be able to choose to view the post if they wish. You must come up with a sensible way to implement this.
- In addition to the normal functionality, only administrators may view all reported posts. Reported posts should be displayed in a logical manner together with the reason(s) they were reported.
- An administrator can then decide whether to delete the post or remove the report(s) associated with it.
    - When a post is deleted by an administrator, the same behaviour should apply as when a user deletes their own post.
    - ***BONUS:*** *alert a user whose post has been deleted via an appropriate notification. You must come up with an aesthetically pleasing way to integrate this into the website.*

- ***BONUS****: come up with a way to detect users who are abusing the "report post" functionality. To obtain these marks, you will have to come up with a sensible system for assessing the situation and a user-friendly way for admins to deal with such users.*

Search

The website must provide search functionality for the following:

- Other users by name, username, or email address.
- Posts by description
- Albums by name and description
- Hashtags
- ***BONUS**: The search functionality must be able to find something **even if the search term is a little incomplete or spelled incorrectly**, i.e., fuzzy string searching: the technique of finding strings that match a pattern approximately (rather than exactly).*

  ***Note**: this does **not** refer to simply adding wildcards to your searches. A fuzzy search should be able to find words even when letters are missing, swapped, or misspelled.*

- ***BONUS**: Add **autocomplete** functionality that suggests results as the user types. You would need to come up with an intuitive design for this.*

Search results must be interactive. Where appropriate, they should provide links to the relevant pages, such as user profiles or posts, and hashtags should remain clickable.

User Roles

The Unregistered User

An unregistered user can only do the following:

- Register as a new user or log in using the splash page. The splash page must not provide any other functionality.
  Note: An unregistered user cannot interact with the website in any other way.

The Registered User

A registered user can perform the following actions:

- Log in and log out.
- View the local activity feed (their own activity and that of their friends) as well as the global activity feed (activity from all users).
- Search for posts or users and view other users' profile pages.
- Edit the appropriate information on their own profile.
- Add new posts.
- Manage (edit/delete) their own posts (this is discussed further under Posts).
- Delete their own profile. Deleting their profile removes their user account from the database.

User Interaction

A registered user can perform the following interactions with other users:

- Connect with other users by sending friend requests.
- View the complete profiles of their friends.
  Note: When a user is not friends with another user, they may only view that user's name and profile picture.
- Unfriend other users.
- Accept or decline incoming friend requests.

The Admin

Some users must be able to log in as administrators. Administrators have all the functionality available to registered users.

Additionally, they can do the following:

- Manage (edit/delete) all activity.
- Manage (edit/delete) all posts.
- Manage (edit/delete) all user accounts.
- Add new reasons for reporting posts.

- ***BONUS**: add functionality for admins to **verify accounts**. A (non-admin) user should only be able to request that their account be verified if their account is more than one week old, if they have created at least two posts, and left at least 5 comments. Admins should have access to a list of verification requests that they can approve/deny on a case-by-case basis. Once an account has been verified, it should be clearly indicated on the user's profile in a visually meaningful way.*

Usability

- *Design a website that is **intuitive, consistent, and easy to use**. It is your responsibility to create logical and user-friendly interactions throughout the website.*
- *For example:*
  - *A log out button must not be visible when a user is not logged in, and a log in button must not be visible when a user is already logged in.*
  - *When a user **creates** an account, they must **automatically be logged in** with that account.*
  - ***All form fields must have working labels**, i.e., when you click on the label, focus is given to its accompanying input.*
  - ***Well-designed error messages should appear when appropriate**, e.g., when a user enters incorrect login details.*
  - ***Navigation must stay consistent** throughout the website and must provide a **logical way to access the core functionality**.*
  - *Text that is interactive must **look interactive** (buttons, links, hashtags, etc.).*
  - ***Clicking on the website logo/name must take the user to the home page**.*

- ***Tip**: Test the usability of your site by asking other people (especially non-IT students) to interact with it to find out how easy it is for them to navigate.*
- ***Marks may be deducted for poor design decisions that negatively affect the user experience, such as leaving debugging alerts (or other unnecessary alerts) in the final submission.***
- ***BONUS**: allow the user to toggle between a standard (light mode) and a dark theme (night mode).*

Visual Design

- Your website must have a **professional and visually appealing design**. Refer to well-designed websites for inspiration, e.g., https://www.awwwards.com, https://dribbble.com/shots/popular/web-design, etc.
- **Use a consistent colour palette that complements your website's overall design**, e.g. browse the colour lovers website (https://www.colourlovers.com/) or use Adobe Color (https://color.adobe.com/) or Coolors (https://coolors.co/) to find colour palettes and patterns or search online for colour scheme creators.
- All form elements must be styled to match your website's visual theme. **Default HTML or Bootstrap form controls** (such as buttons and input fields) **may not be used** without appropriate custom styling.
- Your website must use at least two and no more than three fonts. These may **not** be the default HTML, Tailwind CSS, or Bootstrap fonts. Use fonts that look good and fit the overall design of the site. **You must embed fonts for your website. Consider using something like Google Fonts for this.**
  ***Note**: Fonts must be legible. Do not use more than three fonts for the entire website. Do not use stylised fonts for body text. Do not use standard Windows fonts. Embed fonts with CSS / Tailwind CSS.*

Technologies and Techniques

You may only use the following technologies and techniques:

Technologies:

- **Valid JavaScript code everywhere** (no exceptions). This includes any JSX and additional styling / class names that you may include in your application. Ensure that valid HTML is generated when the application is viewed using "View Source", i.e., one <!DOCTYPE>, one <head>, and one <body> element.
- **CSS or Tailwind CSS** for styling, embedding of fonts, etc. Do not use inline or embedded style sheets. Use CSS variables where appropriate. No component libraries (like Material UI / shadcn / ChakraUI) are allowed.
- **CSS cascading variables** for aspects relating to the style identity of your website, e.g., your theme colours and fonts.
- **Embed** a favicon that fits the theme of your website in all your pages.
- You must use GitHub throughout the semester and submit a link to your repository for each deliverable.

- You are permitted to use Bootstrap but you will need to customise the styling of the components. You will have to use either CSS or Tailwind CSS to style these elements according to your website theme. **You will lose marks if any default Bootstrap is styling visible on your website**.
- **JSON** to transfer your data back and forth between client and server (for example, during fetch calls).
- At least one instance of a Promise for asynchronous behaviour (this can include the built-in async behaviour of using fetch). No third-party request plugins (Axios, ky) are allowed, and you should use the native Fetch API.
- ***BONUS****: demonstrate branching in GitHub by having different branches for logically separated website content, e.g., having a branch with a README file explaining your project, having a branch dedicated to adding certain features. You must have* ***at least one branch that has been merged back into the main branch*** *to get the bonus.*

Techniques:

- **Responsive web design with at least two breakpoints.**
  Note*: Your website must display correctly on any resolution.*
- **Error-free** React and Node.js code. Sensible coding, you must only connect to your database in one place. Keep your code DRY (**D**on't **R**epeat **Y**ourself).
  - Separate your code into reusable components. For example, one component for an add/edit form, one for displaying a post summary, one for displaying an album card, or one for a context menu.
- **Session / cookie management**.

Database

- You are permitted to use MongoDB Atlas, hosting your own instance of MongoDB is optional.
- There must be **LOGICAL** test data in the database for demo purposes.
- There must be *at least* the following:
  - 10 posts, including descriptions.
  - 5 albums.
  - 8 registered users.

  NB: for general testing purposes, you must have one regular user account with at least 1 post, 1 album, and 2 friends. They must have the following login details:
  Email address: *test@test.com*
  Password: *test1234*
- For the purposes of testing and marking, passwords should not be hashed.
- **Remember to export your database and include it in your final submission.**

Directory Structure

- Structure all your files into a logical directory structure.
- Make use of correct file naming conventions.
- Use **relative file paths** for references.
- Put images, fonts, CSS files in their respective folders.

Completing the Project

You have until 29 October to complete the project.

You have to submit on ClickUP. **Remember to download and double-check your submissions. Late submissions / submissions in the incorrect place will not be accepted.**

To avoid running into problems on 2 & 3 November, it is crucial that you test your project by running the Docker image locally throughout the semester. Ideally, you should test that everything still works each time you add new functionality. **Avoid blind-coding at all costs.** Do not wait until just before the submission deadlines to start testing your project.

Make sure to retest the entire site after submitting to ensure that what you will be assessed on is fully functional.

Docker - Things to Look out for

- Your **entire application** should be Dockerised, this includes the frontend and backend code.
- On marking day, the markers will download your submission and run the relevant commands to run your Docker image.
- Ensure that you test your application frequently and thoroughly using Docker. Some things can differ between running your application locally (outside of Docker) and running it using Docker.
- Do not wait until submitting to learn how to Dockerise your project as you will **not be marked** unless you use it.

Mark breakdown

Deliverable 0 - Wireframe: 10%

Deliverable 1 - Front-end: 15%

Deliverable 2 - Back-end 20%

Final Project (Deliverable 3): 55%

Start Today!